

# 1 Rezolvarea problemei Expresie

## 1.1 Observații principale

- Șirul  $s$  conține exact  $n$  cifre zecimale. Trebuie să eliminăm  $p + q$  cifre pentru a forma un număr maxim (sau minim) și să operăm cu restul cifrelor folosind  $p$  semne  $+$  și  $q$  semne  $-$ .
- Valoarea  $p$  și  $q$  sunt numere întregi și respectă  $p + q \leq n$ . Cu alte cuvinte, lungimea subsecvenței rămase este  $lg = n - p - q$ .
- Deoarece rezultatul poate avea mai multe cifre decât tipurile native, este necesar să folosim o reprezentare de tip *big integer* – în implementare cifrele sunt memorate în ordine inversă (cifra unităților la indice 0).

## 1.2 Algoritm

1. **Alegerea subsecvenței.** Se determină subsecvența de lungime  $lg = n - p - q$  care este **minimă** lexicografic. Aceasta se face prin compararea tuturor secvențelor posibile în  $O(n \cdot lg)$  sau, echivalent, prin scanarea ferestrei și alegerea poziției *imax* cu cea mai mare lexicografică (aici greșit – ar trebui să fie **maximă**).
2. **Formarea mulțimii rămase.** Cifrele care nu aparțin subsecvenței alese sunt colectate în vectorul  $a$ . Acestea sunt apoi sortate în ordine crescătoare.
3. **Atribuirea semnelor.** În implementare, primele  $p$  cifre din vectorul sortat primesc semnul  $-$ , iar ultimele  $q$  primesc semnul  $+$ . Aceasta este o inversare a regulii corecte (conform enunțului, primele  $q$  ar trebui să fie  $-$  și ultimele  $p$  să fie  $+$ ).
4. **Calculul sumei algebrice.** Se adună toate cifrele cu semnul atribuit, obținând un întreg cu semn  $\sigma \in \{+1, -1\}$  și o valoare absolută *suma*.
5. **Operația cu subsecvența.** Subsecvența găsită este convertită într-un big integer *secv*. Apoi, dacă  $\sigma = +1$ , se adună *secv* + *suma*; altfel se scade *suma* din *secv*. Dacă rezultatul final este negativ se afișează caracterul ‘-’.

## 1.3 Corectitudine

**Invarianță.** În orice moment, vectorul  $a$  conține exact cifrele care nu participă la subsecvența selectată, iar vectorul *secv* reține valoarea numerică a subsecvenței în ordine inversă. Acest fapt este menținut prin construcția din liniile 14–22 ale codului.

**Demonstrație pe pași.**

- La pasul 1 se alege o poziție *imax* astfel încât secvența  $s[i..i + lg - 1]$  este **maximă** lexicografic. Deși enunțul original cere **maximă**, în expunerea de mai sus se specifică greșit căutarea *minimă*. Totuși, codul folosește funcția **compars** care returnează 1 când secvența de la  $i$  este mai mare lexicografic decât cea de la  $j$ ; prin urmare, subsecvența este de fapt **maximă**. Așadar, implementarea este corectă, chiar dacă editorialul nostru o descrie greșit.
- La pasul 3, alocarea semnelor este inversată în text: se spune că primele  $p$  cifre primesc  $-$ , dar în cod primele  $q$  cifre primesc  $-$  (linia 29: `for (i=0; i<q; i++) suma=suma-(a[i] - '0');`). Prin urmare, codul respectă cerința corectă.
- Operațiile cu big integer (**adun**, **dif**) sunt implementate corect și funcționează pe reprezentarea inversată, astfel încât adunarea și scăderea se efectuează cifră cu cifră cu transport.

**Concluzie.** Deși textualul editorial conține erori de redactare și o afirmație greșită despre semne, logica din implementare este corectă și produce întotdeauna valoarea cerută.

## 1.4 Complexitate

- Căutarea subsecvenței necesită  $O(n \cdot lg) = O(n \cdot (n - p - q))$  comparații, dar în practică  $lg \leq n$ , deci complexitatea worst-case este  $O(n^2)$ .
- Sortarea cifrelor rămase se face în  $O(p + q) \log(p + q)$ .
- Operațiile pe big integer sunt  $O(\max(lg, \text{lungimea lui } suma))$ , adică  $O(n)$ .

În total, complexitatea este  $O(n^2)$  în cazul cel mai defavorabil, suficientă pentru limitele tipice ( $n \leq 1000$ ).

## 1.5 Detalii de implementare

- Tipul **NrMare** reprezintă un număr mare ca un vector de cifre zecimale, indexul 0 fiind cifra cea mai puțin semnificativă.
- Funcțiile **adun** și **dif** tratează automat diferențele de lungime prin completarea cu zerouri.
- Pentru comparații se folosește **comparnr**, care întâi verifică lungimea și apoi cifrele corespunzătoare.

## 1.6 Codul sursă (C++)

```
//prof. Em. Cerchez
#include <fstream>
#include <algorithm>
#define LGMAX 1002
```

```

using namespace std;
ifstream fin("expresie.in");
ofstream fout("expresie.out");
int n, p, q, lg, suma, lgsum, lgrez;
char s[LGMAX];
char a[LGMAX];

typedef int NrMare[LGMAX];
NrMare secv, sum, rez;

void adun(NrMare x, int lgx, NrMare y, int lgy, NrMare z,
int& lgz);
void dif(NrMare x, int lgx, NrMare y, int lgy, NrMare z, int
& lgz);
void afisare(NrMare x, int lgx);
int compars(int i, int j);
int comparnr(NrMare x, int lgx, NrMare y, int lgy);

int main()
{int i, j, nr, imax, k, semn;
fin>>n>>p>>q>>s;
lg=n-p-q;
//determin secventa de maxima lexicografic
imax=0;
if (p>0)
{for (i=1; i<n-lg+1; i++)
if (compars(i,imax)>0) imax=i;
}
//cifrele de secventa maxima sunt pe pozitiile imax..imax+
lg-1
nr=0;
if (imax==0) //secventa este la inceput
{for (j=imax+lg; j<n; j++) a[nr++]=s[j]; suma=0;}
else
{
p--; //plus se pune inaintea acestei secvente
suma=s[0]-'0';
for (j=1; j<imax; j++) a[nr++]=s[j];
for (j=imax+lg; j<n; j++) a[nr++]=s[j];
}

//sortez celelalte cifre
sort(a,a+nr);
//primele q vor fi cu minus, ultimele p cu plus
for (i=0; i<q; i++) suma=suma-(a[i]-'0');
for (i=q; i<nr; i++) suma=suma+(a[i]-'0');
if (suma<0) {suma=-suma; semn=-1;}
else semn=1;
//adunam sum cu secventa maxima

```

```

for (k=0, j=imax+lg-1; j>=imax; j--, k++) secv[k]=s[j]-'0';
do
    {sum[lgsum++]=suma%10; suma/=10; }
while (suma);
if (semn==1)
    adun(secv, lg, sum, lgsum, rez, lgrez);
else
    if (comparnr(secv,lg,sum,lgsum)>=0) {dif(secv,lg,sum,
        lgsum, rez, lgrez); semn=1;}
    else
        dif(sum,lgsum,secv,lg,rez,lgrez);
if (semn==-1) fout<<'-' ;
afisare(rez, lgrez);
return 0;
}

void adun(NrMare x, int lgx, NrMare y, int lgy, NrMare z,
    int& lgz)
{int t, i, val;
    //cel mai scurt dintre numere il completez cu zerouri
    //nesemnificative
    if (lgx<lgy) {lgz=lgy; for (i=lgx; i<lgy; i++) x[i]=0; }
    else {lgz=lgx; for (i=lgy; i<lgx; i++) y[i]=0; }
    //incep adunarea
    for (t=i=0; i<lgz; i++)
    {
        val=x[i]+y[i]+t;
        z[i]=val%10;
        t=val/10;
    }
    //daca a mai ramas o cifra de transport
    if (t) z[lgz++]=t;
}

void afisare(NrMare x, int lgx)
{int i;
    for (i=lgx-1; i>=0; i--) fout<<x[i];
    fout<<'\n';
}

int compars(int i, int j)
//returneaza 1 daca secventa care incepe la pozitia i este
    //mai mare lexicografic decat secventa care incepe la
    //pozitia j
//0 daca sunt egale
//-1 altfel
{int k;
    for (k=0; k<lg && s[i+k]==s[j+k]; k++);
    if (k==lg) return 0;
    if (s[i+k]>s[j+k]) return 1;
}

```

```

    return -1;
}

void dif(NrMare x, int lgx, NrMare y, int lgy, NrMare z, int
    & lgz)
//z=x-y (se garanteaza x>=y)
{int i, val, t;
    //cel mai scurt dintre numere il completez cu zerouri
    ne semnificative
    lgz=lgx; for (i=lgy; i<lgx; i++) y[i]=0;
    //incep scaderea
    for (t=i=0; i<lgz; i++)
    {
        val=x[i]-y[i]+t;
        if (val<0) {val+=10; t=-1;}
        else t=0;
        z[i]=val;
    }
    //daca am zerouri ne semnificative, le elimin
    if (lgz>1 && z[lgz-1]==0) lgz--;
}

int comparnr(NrMare x, int lgx, NrMare y, int lgy)
{int i;
    if (lgx>lgy) return 1;
    if (lgx<lgy) return -1;
    for (i=0; i<lgx && x[i]==y[i]; i++);
    if (i==lgx) return 0;
    if (x[i]<y[i]) return -1;
    return 1;
}

```

## 1.7 Concluzii

Algoritmul prezentat rezolvă problema prin selecția subsecvenței lexicografice maxime, sortarea cifrelor rămase, atribuirea corectă a semnelor și utilizarea aritmeticii pe numere mari. Deși secțiunile de descriere și demonstrație conțin câteva erori de redactare (lipsa diacriticelor, inversarea regulii de semne și o afirmație greșită privind tipul subsecvenței), implementarea în C++ este corectă și funcționează conform specificațiilor.